SECURITY · AUDIT

CRM Sync — Security Audit & Paired Data Requirements

The security audit and paired-data requirements: which data must travel together, and the controls that keep every pair consent-qualified.

Date: **2026-05-18** Version: **1.1** Worker Version: **dac8f178-f6ed-4a11-96be-f640a67c64ae**

1. SECURITY AUDIT SUMMARY

1.1 Route Auth Coverage (Post-Hardening)

All 67 routes verified. Every write endpoint and admin endpoint now requires authentication. Per-tenant admin keys provide granular access control with platform key fallback.

AUTH LAYERS

LAYER	MECHANISM	PROTECTS
Bearer Token	`Authorization: Bearer <ADMIN_KEY>` or per-tenant `admin_key`	All `/admin/*`, `/sync/*`, `/config POST`, `/tags/create`, `/webhooks/upsell`, `/platform/config`
Admin Key (query)	`?key=<ADMIN_KEY>` in URL	`/setup`, `/onboarding`, `/onboarding/setup`, `/settings`
JWT (user session)	`httpOnly` cookie, HS256 signed	`/auth/me`, `/auth/profile`, `/auth/delete-account`, `/ucp/*`, `/tags/*`, `/segment/*`
HMAC-SHA256	`X-Shopify-Hmac-SHA256` header	`/webhooks/customer-update`, `/webhooks/customer-create`, `/gdpr/*`, `/api/webhooks`
OAuth State	UUID-keyed KV nonce (`oauth_state:{uuid}`), single-use, includes shop	`/auth/callback`, `/auth/webflow/callback`, `/auth/google/callback`, `/auth/shopify/callback`
Cloudflare Access	OTP email verification	Worker admin URLs (browser access)

COMPLETE ROUTE MATRIX

#	METHOD	PATH	AUTH	TYPE
1	GET	`/health`	None (public)	Health check
2	POST	`/config`	Bearer token	Admin
3	GET	`/config`	None (secrets masked)	Read-only
4	GET	`/admin/tenants`	Bearer token	Admin
5	GET	`/auth/install`	None (OAuth initiation)	OAuth
6	GET	`/auth/callback`	OAuth state verification	OAuth
7	GET	`/embed/footer`	None (public embed)	Embed
8	GET	`/embed/compliance`	None (public embed)	Embed
9	GET	`/embed/account`	None (public embed)	Embed
10	GET	`/embed/dashboard`	None (public embed)	Embed
11	GET	`/setup`	Admin key (query)	Admin UI
12	GET	`/settings`	Admin key (inside handler)	Admin UI
13	GET	`/onboarding`	Admin key (query)	Admin UI
14	GET	`/onboarding/setup`	Admin key (query)	Admin UI
15	POST	`/onboarding/auto-setup`	Bearer token	Admin
16	GET	`/auth/webflow/connect`	None (OAuth initiation)	OAuth
17	GET	`/auth/webflow/callback`	OAuth state verification	OAuth
18	GET	`/api/xano/actions`	None (public manifest)	Read-only
19	POST	`/auth/signup`	None (registration)	Auth
20	POST	`/auth/login`	None (login)	Auth
21	GET	`/auth/me`	JWT (inside handler)	User

#	METHOD	PATH	AUTH	TYPE
22	POST	`/auth/logout`	None (clears cookie)	Auth
23	GET	`/auth/google/login`	None (OAuth initiation)	OAuth
24	GET	`/auth/google/callback`	OAuth state verification	OAuth
25	GET	`/auth/shopify/login`	None (OAuth initiation)	OAuth
26	GET	`/auth/shopify/callback`	PKCE verification	OAuth
27	POST	`/auth/profile`	JWT (inside handler)	User
28	POST	`/auth/delete-account`	JWT (inside handler)	User
29	POST	`/auth/forgot-password`	None (email-based)	Auth
30	GET	`/auth/reset-password`	Token in URL	Auth
31	POST	`/auth/reset-password`	Token in body	Auth
32	POST	`/auth/consent-sync`	None (see note)	Consent
33	GET	`/ucp/consent-history`	JWT (inside handler)	User
34	POST	`/ucp/tags`	JWT (inside handler)	User
35	POST	`/ucp/translate`	JWT (inside handler)	User
36	POST	`/segment/search`	JWT (inside handler)	User
37	GET	`/segment/stats`	JWT (inside handler)	User
38	POST	`/segment/count`	JWT (inside handler)	User
39	POST	`/webhooks/customer-update`	HMAC-SHA256	Webhook
40	POST	`/webhooks/customer-create`	HMAC-SHA256	Webhook
41	POST	`/webhooks/webflow-item-changed`	Webflow webhook	Webhook
42	POST	`/api/webhooks`	HMAC-SHA256	Webhook

#	METHOD	PATH	AUTH	TYPE
43	POST	`/gdpr/customer-redact`	HMAC-SHA256	GDPR
44	POST	`/gdpr/data-request`	HMAC-SHA256	GDPR
45	POST	`/gdpr/shop-redact`	HMAC-SHA256	GDPR
46	GET	`/tags`	JWT (inside handler)	User
47	GET	`/tags/user`	JWT (inside handler)	User
48	POST	`/tags/user`	JWT (inside handler)	User
49	POST	`/tags/create`	Bearer token	Admin
50	POST	`/admin/init-tag-system`	Bearer token	Admin
51	POST	`/admin/xano-schema`	Bearer token	Admin
52	POST	`/admin/xano-reseed`	Bearer token	Admin
53	POST	`/admin/adobe-schema`	Bearer token	Admin
54	POST	`/webhooks/upsell`	Bearer token	Admin
55	POST	`/admin/register-webhooks`	Bearer token	Admin
56	POST	`/admin/webflow-ensure-fields`	Bearer token	Admin
57	POST	`/admin/webflow-sync-test`	Bearer token	Admin
58	GET	`/admin/webflow-test`	Bearer token	Admin
59	GET	`/admin/shopify-customers`	Bearer token	Admin
60	GET	`/admin/shopify-test`	Bearer token	Admin
61	POST	`/sync/customers`	Bearer token	Admin
62	POST	`/sync/webflow`	Bearer token	Admin
63	GET	`/admin/tenants`	Bearer token	Admin

#	METHOD	PATH	AUTH	TYPE
64	GET	`/platform/config`	Bearer token	Admin
65	POST	`/platform/config`	Bearer token	Admin
66	POST	`/admin/provision-region`	Bearer token	Admin
67	POST	`/admin/adobe-schema`	Bearer token	Admin

NOTES

- `/auth/consent-sync` **(#32)**: Called from client-side embed scripts with `user_id` in body. The handler writes to Xano consent_records. This endpoint accepts unauthenticated requests from the embed context — consent changes from the cookie banner and compliance page arrive here. This is by design (the embed runs before the user has a JWT session), but the endpoint should be hardened with rate limiting.
- `GET /config` **(#3)**: Returns config with all secrets masked (first 4 + last 4 chars). No auth required for read, but secrets are never exposed.
- **Embeds (#7-10)**: Public HTML endpoints. Secrets are stripped via `getPublicCrmConfig()` before injection.

1.2 Fixes Applied (2026-05-17)

ROUTE	BEFORE	AFTER
`POST /admin/init-tag-system`	Open	Bearer token
`POST /admin/xano-schema`	Open	Bearer token
`POST /admin/xano-reseed`	Open	Bearer token
`POST /admin/register-webhooks`	Open	Bearer token
`POST /admin/webflow-ensure-fields`	Open	Bearer token
`POST /admin/webflow-sync-test`	Open	Bearer token
`GET /admin/webflow-test`	Open	Bearer token
`GET /admin/shopify-customers`	Open	Bearer token
`GET /admin/shopify-test`	Open	Bearer token
`POST /sync/customers`	Open	Bearer token
`POST /sync/webflow`	Open	Bearer token
`POST /tags/create`	Open	Bearer token

2. PAIRED DATA REQUIREMENTS

2.1 System Pairs — CRM Sync (Webflow-Primary Gateway)

CRM Sync uses Webflow as its primary headless gateway and Shopify as the commerce data source. Data flows bidirectionally between 7 systems.

```
Webflow (Gateway) ←→ Worker ←→ Shopify (Commerce)
                          ↕
                Xano (Source of Truth)
                          ↕
                GA4 / Adobe AEP / Resend
```

2.2 Data Pair Matrix

Each pair defines: what data crosses the boundary, which direction, what trigger, and what security contract.

Thesis — keep the castle and the moat; distribute the gold. The incumbent's mistake is not having walls — it is **hoarding all the gold in one vault behind them.** That vault is the blast radius: one breach takes everything, and the hoarding also locks the value away from the people who need it. This architecture keeps the castle and the moat — **Cloudflare's WAF, DDoS, and edge governance still stand** — but **spreads the gold across many pockets**: PCI in Shopify, PII/consent in Xano, a draft mirror in Webflow, grants that verify with a public key. The result is **security *and* accessibility in the same move** — nothing concentrated to steal, everything reachable to those authorized:

- **The walls stand, but they hoard nothing.** Being *inside* the perimeter grants nothing — every request is re-verified per-action (403 without the right consent/mandate). The castle governs access; it does not store the treasure.
- **No single vault.** The gold sits in many pockets — Shopify (PCI), Xano (PII/consent), a draft Webflow mirror. Breach one and you get a *fragment*, never the hoard.
- **No secret to steal.** Entitlement grants are offline-verifiable with a **public** key — the proof of trust is not a secret an attacker can exfiltrate.
- **No single point to take down.** Three legs, self-healing — the loss of any one leg is a Tuesday, not a breach.

- **Distributed = usable.** Because the gold is spread out **for people and agents to use** (consent-gated), distribution *is* the accessibility model — not a vault to petition, a network to reach.

"Zero Trust" is simply the admission that a hoarded, *trusted* vault is how you get breached.

Substrate ≠ Security. Webflow and Xano are *where this runs* — not *what makes it safe.* The controls live in the architecture below: HMAC-SHA256 mandates, PKCE + OIDC auth, JWT sessions, per-action authorization, offline-verifiable Ed25519 grants, real-time consent. These are the same cryptographic primitives enterprise systems use — and the agentic model (A2A *read* vs AP2 *spend*, per-action gating, revocable consent) is one most enterprise CRMs do not have at all. No-code substrate; enterprise-grade boundary logic. **Judge the boundary logic, not the vendor logos.**

Closed-boundary software and AI governance. A closed, monolithic, single-vendor boundary governs by *containment* — a fixed perimeter, a fixed release cadence, a policy engine you cannot reach into. For human-speed CRM that is tolerable. For governing an autonomous, real-time AI actor it is a liability, because AI incidents move faster than a closed system can respond. A closed boundary denies the three capabilities AI governance requires:

- **It cannot heal.** When state drifts or a record corrupts, a closed system waits for the vendor's next sync, patch, or support ticket — while an agent compounds the error on stale data. Self-healing (idempotent reconcile; the mirror that rebuilds itself from the system of record) is the only thing that keeps an autonomous actor off bad data.
- **It cannot fall back.** A monolith degrades to *off*: if its one plane is down, governance stops — and the agents either halt (lost commerce) or, worse, proceed **ungoverned** (a control gap is an incident). Graceful degradation (authorize offline, capture online, buffer-and-replay across three legs) keeps

the guardrails *on* even when a leg is down. Governance must fail closed **and keep working**.

- **It cannot forward-deploy.** When an incident needs a rule changed *now* — revoke a mandate, tighten a scope, block a rail — a closed boundary makes you file a change request and wait for a release window. An edge-deployed, code-owned governance layer pushes the new rule to every request in seconds. AI incident remediation is measured in seconds, not sprints.
- **It taxes your success.** A closed boundary bills you to move your own data — egress fees that scale with volume. The better your AI performs, the more it queries and moves, the higher the bill: you are taxed *precisely as AI adoption succeeds.* Governance you own, at the edge, moves data without a toll booth — so the economics **improve** as you scale instead of punishing it.

Implication: you cannot govern an autonomous, real-time actor with a system that changes only on the vendor's schedule, recovers only on the vendor's timeline, works only when every component is up, and **charges you more the more the AI works.** AI governance requires a layer you can **heal, fall back, forward-deploy, and scale — without a toll booth — at the speed the AI operates**, which a closed boundary, by definition, forbids. This is the "Risk Management for AI Incident Remediation" framework the architecture names.

Layer model (canonical: the published architecture at https://crm-sync.webflow.io). Five layers, each with a distinct job; no single one is a hard dependency — the three-leg design degrades gracefully if any is unavailable:

1. **Cloudflare — edge governance (the "God layer").** Edge OTP + admin-only gating, WAF, DDoS protection, bot mitigation, secrets, rate limits. Governs security for every request — but is **not a single point of failure**: entitlement grants are offline-verifiable (Ed25519 public key), writes buffer and replay, and the PWA serves cached reads.

2. **Authentication Layer.** Google (OAuth + PKCE), Shopify (OIDC + PKCE), Email/Password (Xano direct).
3. **Verification Layer — Xano.** Find-or-create users; the **system of record** for identity, PII, and consent state.
4. **Token.** Worker-issued JWT (7-day TTL) — the scoped session / mandate credential.
5. **Data Layer.** Webflow (Collection Sync + Entitlement — the draft-only CRM mirror, Pair 2), Shopify (metaobjects, tags, A2A/AP2 JSON — the **PCI plane**: cardholder data stays in Shopify's PCI scope), GA4 + Adobe/BAU (user props, UCP conversions).

Plane separation: payment/cardholder data stays in **Shopify's PCI scope**; identity / PII / consent are the **Xano** system of record; **Webflow** holds a draft-only CRM read-model mirror (no PCI, no Webflow commerce); **Cloudflare** governs but is not depended upon.

PAIR 1: SHOPIFY ↔ XANO (CUSTOMER IDENTITY)

FIELD	SHOPIFY SOURCE	XANO TABLE	DIREC
Email	`customer.email`	`storefront_users.email`	Shop Xano
First Name	`customer.firstName`	`storefront_users.first_name`	Shop Xano
Last Name	`customer.lastName`	`storefront_users.last_name`	Shop Xano
Shopify GID	`customer.id`	`storefront_users.shopify_gid`	Shop Xano
Orders Count	`customer.numberOfOrders`	`storefront_users.number_of_orders`	Shop Xano
Total Spent	`customer.totalSpentV2.amount`	`storefront_users.amount_spent`	Shop Xano
Country	`customer.defaultAddress.countryCodeV2`	`storefront_users.country`	Shop Xano
Tags	`customer.tags`	`user_tag_map` (join table)	Bi-direc
Metafields	`customer.metafields` (crm_*)	Derived from tags	Xano Shop

Security: HMAC-SHA256 on webhooks. Admin token (shpua_) for GraphQL. Token auto-refreshed before 60-min expiry.

PAIR 2: XANO → WEBFLOW (CUSTOMERS CRM MIRROR — RESILIENCE LEG)

The Webflow "Customers" collection is a **read-model CRM mirror / backup** of Shopify customer profiles — the same role Salesforce or HubSpot play (contact PII, consent *state*, and commerce *aggregates* for CRM use). It is one leg of the **three-leg resilience design** (Shopify ↔ Xano ↔ Webflow): if one leg is

unavailable, the customer read-model still exists in the others. Mirror items are written as **drafts** — a back-office copy, not published to the live public site.

Plane separation (why this is a CRM mirror, not a PCI / commerce exposure):

- **Shopify = PCI plane** — cardholder / payment data never leaves Shopify's PCI-compliant scope. We never store, mirror, or transmit card data.
- **Xano = PII / identity / consent** — the system of record.
- **Webflow = CRM mirror + presentation** — the profile read-model only, draft-only. **Webflow's native E-commerce is NOT used** (no products, checkout, orders, or payment data).

FIELD	XANO SOURCE	WEBFLOW CMS FIELD	DIRECTION
Name	`storefront_users.full_name`	`name` (required)	Xano → Webflow
Email	`storefront_users.email`	`email`	Xano → Webflow
First/Last Name	`storefront_users.first_name/last_name`	`first-name`, `last-name`	Xano → Webflow
Provider	`storefront_users.provider`	`provider`	Xano → Webflow
Status	Derived from tags	`status`	Xano → Webflow
Tags	`storefront_users.tags` / `user_tag_map`	`tags`, `tag-refs`	Xano → Webflow
Consent state	`user_claims.*`	`consent-tos/privacy/cookie/marketing`	Xano → Webflow
Commerce aggregates	`storefront_users.*`	`number-of-orders`, `amount-spent`, `country`	Xano → Webflow
Shopify ID	`storefront_users.shopify_gid`	`shopify-customer-id`	Xano → Webflow
Adobe fields	`user_extras.*`	`adobe-ecid/email-hash/sync-status/last-synced/identity-graph-id`	Xano → Webflow

Security: Webflow CMS token (write-scoped, worker-held). Mirror items are **drafts** (off the live site). **No PCI / cardholder data** is ever mirrored — payment data stays in Shopify's PCI plane. The mirror performs **no transactional or commerce function**; it is a CRM read-model backup, comparable to a Salesforce/HubSpot contact sync.

PAIR 3: XANO ↔ GA4 (ANALYTICS EVENTS)

EVENT	DATA SENT	DIRECTION	TRIGGER
`crm_tags_updated`	tags_added, tags_removed, crm_status, crm_tier, crm_segment	Xano → GA4	Tag mutation
`crm_sync`	source, synced count	Xano → GA4	Cron sync
`crm_form_submit`	form_type, email (DataLayer only)	Client → GA4	Form bridge
`crm_upsell`	upsell_source, product_count, total_value	Xano → GA4	Upsell event
User Properties	crm_status, crm_tier, crm_segment, crm_tags, crm_campaign, consent_marketing, consent_tos	Xano → GA4	Any mutation

Security: GA4 API Secret stored in KV config. No PII sent — only tag categories and consent state. Client ID format: `crm-sync.{userId}`.

PAIR 4: XANO ↔ ADOBE AEP (CDP STREAMING)

FIELD	XANO SOURCE	XDM PATH	DIRECTION	TRIGGER
Email (hashed)	SHA-256(`email`)	`identityMap.Email[0].id`	Xano → AEP	Customer update
Phone (hashed)	SHA-256(`phone`)	`identityMap.Phone[0].id`	Xano → AEP	Customer update
Name (hashed)	SHA-256(`name`)	`_shopifyCrmSync.hashedName`	Xano → AEP	Customer update
Consent	Derived from tags	`consents.marketing.email/push` , `consents.adID` , `consents.personalize`	Xano → AEP	Tag mutation
Subscriptions	Derived from tags	`_shopifyCrmSync.subscriptions.{type}`	Xano → AEP	Form bridge
Commerce	Product list	`commerce.productListAdds` , `productListItems[]`	Xano → AEP	Upsell event
ECID	AEP response	`user_extras.adobe_ecid`	AEP → Xano	Sync result
Sync status	Sync result	`user_extras.adobe_sync_status`	Worker → Xano	After push

Security: Adobe IMS OAuth (client_credentials). PII hashed with SHA-256 via Web Crypto API before transmission. Raw PII never leaves the worker. IMS tokens cached in KV with TTL.

PAIR 5: XANO ↔ RESEND (TRANSACTIONAL EMAIL)

EMAIL TYPE	DATA SENT	DIRECTION	TRIGGER
Welcome	email, name, set-password link (24h token)	Xano → Resend	New Shopify-origin user
Password Reset	email, reset link (1h token)	Xano → Resend	Forgot password

Security: Resend API key stored as wrangler secret. Reset tokens are KV-stored with TTL, single-use.

PAIR 6: WORKER ↔ CLOUDFLARE KV (CONFIG & STATE)

KEY PATTERN	DATA	SENSITIVITY	TTL
`tenant:{shop}:config`	Full CrmSiteConfig with all credentials + admin_key	HIGH	Persistent
`tenant:{shop}:tag_table_ids`	Xano table IDs (crm_tags + user_tag_map)	LOW	Persistent
`tenant:{shop}:webflow_tags_collection_id`	Webflow CRM Tags collection ID	LOW	Persistent
`tenant:{shop}:sync:customers:last_run`	ISO timestamp of last cron sync	LOW	Persistent
`tenants:index`	`TenantEntry[]` (shop, region, registered_at)	LOW	Persistent
`platform:config`	Shared platform credentials	HIGH	Persistent
`adobe_token:{shop}`	Adobe IMS access token	HIGH	~24h
`pkce:{state}`	PKCE code_verifier	MEDIUM	5 min
`oauth_state:{uuid}`	OAuth state + shop (UUID-keyed, no global collisions)	MEDIUM	10 min
`reset:{token}`	Password reset metadata	MEDIUM	1h / 24h

Security: KV encrypted at rest (Cloudflare managed). Secrets masked in GET /config. Short TTLs on auth state.

2.3 Baseline Data Contract

Every data pair has the following contract:

1. **Identity resolution** — Email is the primary key across all systems (Shopify GID for Shopify-specific operations)
2. **Source of truth** — Xano is canonical. Conflicts resolved by most-recent-write-wins.

3. **PII boundary** — Raw PII stays within Worker + Xano + Shopify + Webflow CMS + Resend. GA4 and Adobe AEP receive hashed or categorized data only.
4. **Auth boundary** — Every cross-system call uses the target system's native auth (Shopify Admin token, Webflow CMS token, Xano API key, GA4 API secret, Adobe IMS OAuth, Resend API key).
5. **Tenant isolation** — Config and credentials are scoped to `tenant:{shop}:config`. No cross-tenant reads.
6. **Audit trail** — All consent mutations logged to `consent_records` (append-only). Adobe sync logged to `adobe_sync_log`.

---

3. REMAINING HARDENING ITEMS

#	ITEM	PRIORITY	STATUS
1	Rate limit `/auth/consent-sync`	Medium	Open
2	Rate limit `/auth/signup` and `/auth/login` (brute force)	Medium	Open
3	Rate limit `/auth/forgot-password` (email enumeration)	Medium	Open
4	Add `ADMIN_KEY` secret if not already set	Critical	Verify
5	Rotate Shopify refresh tokens before 90-day expiry	Low	Automated (cron)
6	Verify Cloudflare Access policy covers all admin URLs	Low	Verify
7	Add CSP headers to embed HTML responses	Low	Open
8	Add `X-Content-Type-Options: nosniff` to all JSON responses	Low	Open